

অপারেটিং সিস্টেমের ভূমিকা
(Introduction to Operating Systems)
প্রফেসর চেস্টার রেবেরো

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ
(Department of Computer Science and Engineering)

ভারতীয় প্রৌদ্যোগিকী সংস্থান, মাদ্রাজ
(Indian Institute of Technology, Madras)

সপ্তাহ – 01 বক্তৃতা- 13

xv6 এ প্রসেস পরিচালনার জন্য সিস্টেম কল

পূর্ববর্তী ভিডিওতে, আমরা বিভিন্ন প্রসেস পরিচালনার সম্পর্কিত গুরুত্বপূর্ণ সিস্টেম কল দেখেছি। সুতরাং, আমরা এই সিস্টেম কল যেমন ফর্ক, exec, wait এবং exit দেখেছি। তাই এই ভিডিওতে, আমরা দেখব কিভাবে এই সিস্টেম কলগুলি অপারেটিং সিস্টেমের মধ্যে বাস্তবায়িত হয়।

(স্লাইডটাইপ পড়ুন: 00:38)

তাই বিশেষভাবে, আমরা দেখব কিভাবে সিস্টেম কলগুলি OS xv6 এ প্রয়োগ করা হয়েছে। তাই, আমরা xv6 উৎস কোড উল্লেখ করব, যা এই বিশেষ ওয়েবসাইট থেকে একটি পুস্তিকা আকারে ডাউনলোড করা যাবে। অনুগ্রহ করে সংশোধন সংখ্যা ৪ ডাউনলোড করুন, যাতে এটি নির্দিষ্ট ভিডিওর সাথে মেলে।

(স্লাইডসময় পড়ুন: 01:00)

তাই আসুন শুরু করি ফর্ক সিস্টেম কল দিয়ে। পূর্ববর্তী ভিডিওতে, আমরা দেখেছি যে যখন ফর্ক সিস্টেম কল আসে তখন এটি একটি চাইল্ড প্রসেস তৈরি করে। এখানে চাইল্ড প্রসেসে ফর্কের রিটার্ন মান pid এর মান হবে 0; ফলস্বরূপ, এই সবুজ লাইনটি চাইল্ড প্রসেস দ্বারা একচেটিয়াভাবে চালানো হয়। প্যারেন্ট প্রসেসের মধ্যে, ফর্ক দ্বারা প্রদত্ত মানটি 0 এর চেয়েও বেশি হবে, প্যারেন্ট ফর্ক দ্বারা প্রদত্ত মানটি চাইল্ড প্রসেসের pid ভ্যালু হবে, সুতরাং এই বেগুনি লাইনগুলি প্যারেন্ট প্রসেস দ্বারা সঞ্চালিত হবে।

(স্লাইডসময় পড়ুন: 01:42)

অপারেটিং সিস্টেমের ভিতর, ফর্কটি একটি নতুন প্রসেস নিয়ন্ত্রণ ব্লক তৈরি করে এবং এটি পূরণ করে। xv6- এ, প্রসেস নিয়ন্ত্রণ ব্লক বা PCB- একটি struct proc দ্বারা সংজ্ঞায়িত করা হয় যা এখানে দেখানো হয়েছে। সুতরাং, ফর্কটি তৈরি করা নতুন চাইল্ড প্রসেসের সাথে সংশ্লিষ্ট struct proc এর বিভিন্ন উপাদান পূরণ করে। আবার মনে করুন xv6 অপারেটিং সিস্টেমে সংজ্ঞায়িত করা একটি p টেবিলও। p টেবিল procs এর একটি অ্যারে ধারণ করে, অ্যারের আকার হল NPROC যা মোট প্রসেস সংখ্যা, xv6 OS- এ একক সময়ে সঞ্চালন করতে পারে। সুতরাং, প্রতিটি প্রসেসের জন্য এই বিশেষ ptable এ একটি এন্ট্রি বরাদ্দ করা হয়।

(স্লাইডটাইপ পড়ুন: 02:34)

xv6 তে ফর্ক বাস্তবায়ন এখানে দেখানো হয়। তাই, এটি লাইন সংখ্যা 2554-এ সোর্স কোড তালিকাতে দেখা যাবে। সুতরাং আপনি এখানে প্রথম লক্ষ্য করবেন যে আমরা 'np' নামে একটি পয়েন্টার ঘোষণা করছি যা struct proc পয়েন্টার হিসাবে সংজ্ঞায়িত করা হয়। সুতরাং, এটি একটি নতুন প্রসেসের অনুরূপ একটি পয়েন্টার তৈরি করা হয়েছে বা নতুন চাইল্ড প্রসেস। এখন ফর্কটি প্রথম ধাপে allocproc কে ডাকে। সুতরাং, allocproc কি করে অর্থাৎ এই ফাংশনটি কি করে তা হল এটি ptable মাধ্যমে পার্স এবং একটি proc কাঠামো যা অব্যবহৃত খুঁজে যাচ্ছে; একবার এই proc কাঠামো পাওয়া যায়, তারপর এটি স্টেট হিসাবে EMBRYO সেট করে। সুতরাং, xv6 EMBRYO- এর মানে হল একটি নতুন প্রসেস যা এখনও এক্সিকিউট করা হয় নি, যা সেট করা হয় তা হল নতুন সন্তানের প্রসেসের জন্য pid। Allocproc মধ্যে সম্পন্ন করা হয় নতুন চাইল্ড প্রসেসের কার্নেল স্ট্যাকের বরাদ্দ এবং কার্নেল স্ট্যাক পূরণ কয়েকটি জিনিসের সঙ্গে যেমন trapframe পয়েন্টার, trapret এবং কন্টেম্পট হিসাবে।

(স্লাইডটাইপ পড়ুন: 03:52)

ফর্ক বাস্তবায়নের পরবর্তী ধাপ হল এটি, এই ফাংশনটি কপিউইম(copyuvm) নামে ডাকুন। সুতরাং, copyuvm ফাংশনটি হল যে এটি পেজ ডিরেক্টরিটি প্যারেন্ট প্রসেস থেকে চাইল্ড প্রসেসে প্রতিলিপি করে। এই copyuvm দুটি প্যারামিটার নেয়; এটি প্যারেন্ট পেজ ডিরেক্টরিটি গ্রহণ করে যা প্রতিনিধিত্ব করা হয় proc পয়েন্ট pgdir; এবং 'sz' দ্বারা উপস্থাপিত প্যারেন্ট এর আকার, এবং এই ফাংশনটি ফেরত দেয় নতুন প্রসেসগুলির পেজ নির্দেশিকাটির একটি পয়েন্টার। এখন আমরা এই বিশেষ ফাংশনকে পরবর্তী স্লাইডে বিস্তারিতভাবে দেখতে পাব, তবে এখন দেখুন যে যদি এই ফাংশনটি ব্যর্থ হয় তবে সবকিছু উল্টে যায়। প্রথমত, কার্নেল স্ট্যাক, যা পূর্বে allocproc এ বরাদ্দ করা হয়েছে তা মুক্ত হয়, এবং np kstack এর মানটি 0 তে নির্ধারণ করা হয় এবং স্টেটটিকে আবার UNUSED এ সেট করা হয়। মনে রাখবেন যে allocproc স্টেটকে EMBRYO তে সেট করেছিল, এখন এখানে

স্টেটটি এখানে অব্যবহৃত বলে সেট করা হয়েছে। তারপর ফর্ক -1 কে ফিরিয়ে দিচ্ছে। সুতরাং, এই -1 ব্যবহারকারী প্রসেসে ফিরিয়ে পাঠানো হয় যা প্রসেস ফর্ককে আহ্বান করেছে।

(স্লাইডটাইপ পড়ুন: 05:15)

ফর্ক বাস্তবায়নের পরের ধাপটি চাইল্ডের সামনে প্যারেন্টের কিছু প্যারামিটার কপি করা; এই পদক্ষেপগুলির মধ্যে সবচেয়ে গুরুত্বপূর্ণ হল তৃতীয় পদ্ধতি যেখানে প্যারেন্ট প্রসেসের সম্পূর্ণ trapframe চাইল্ড প্রসেস এর trapframe মধ্যে কপি করা হয়। এখন মনে রাখা যে trapframe ব্যবহার করা হয় বা পরিবর্তিত হয় যখন একটি হার্ডওয়্যার ইন্টারাপ্ট বা একটি সিস্টেম কল আহরণ করা হয়।

(স্লাইডটাইপ পড়ুন: 05:46)

ফর্ক সিস্টেম কল আহরণ করা হলে যদি আমরা এখানে ফিরে যাই, এটি অপারেটিং সিস্টেমকে এক্সিকিউট করে এবং কার্নেল স্ট্যাকে এই প্রসেসটির জন্য trapframe তৈরি করতে ড্রিগার করে। এখন trapframe ব্যবহার করা হয়, যাতে ফর্ক সিস্টেম কল অপারেটিং সিস্টেমে এক্সিকিউশন সমাপ্ত হলে এটি এই বিন্দুতে ফিরে আসবে। এখন প্যারেন্ট প্রসেসের পুরো trapframe চাইল্ড প্রসেসে অনুলিপি করে, এটি এই প্রসেস থেকে চাইল্ড প্রসেসটি এক্সিকিউট করা অব্যাহত রাখবে; এইভাবে যখন ফর্কটি চাইল্ড প্রসেসের মধ্যে ফেরত আসে, চাইল্ডের প্রসেসটিও এই বিন্দু থেকে চালানো হবে। এছাড়াও, মনে রাখবেন যে প্যারেন্ট প্রসেসের রিটার্ন এবং চাইল্ড প্রসেসের রিটার্নের মধ্যে পার্থক্য হল যে চাইল্ড প্রসেসের pid ভ্যালু 0; যখন প্যারেন্ট প্রসেসের ক্ষেত্রে এটি 0 এর চেয়ে বড় হয়। তাই, আমরা পরবর্তীতে দেখতে পাব কিভাবে এটি অর্জন করা যায়।

(স্লাইডসময় পড়ুন: 06:49)

সুতরাং ফর্ক বাস্তবায়নে, আমরা দেখতে পাচ্ছি যে নতুন প্রসেসের trapframe এ "eax" এর মান 0 সেট করা আছে। সুতরাং, এটি নিশ্চিত করে যে চাইল্ড প্রসেসের, ফর্কের রিটার্ন এর মান 0 সেট করা আছে। আমরা পরে দেখব কিভাবে প্যারেন্ট প্রসেসের মধ্যে ফেরত মান সন্তানের pid তে সেট করা হয়।

(স্লাইডটাইপ পড়ুন: 07:17)

ফর্ক বাস্তবায়নের এই অংশে, অন্য বিষয়গুলি প্যারেন্ট প্রসেস থেকে চাইল্ড প্রসেসে অনুলিপি করা হয়। অন্যান্য জিনিসগুলির মধ্যে এক্সিকিউটেবল নাম, cwd যা বর্তমান কার্যকরী ডিরেক্টরি এবং প্যারেন্ট থেকে ফাইল পয়েন্টারগুলির অনুলিপি।

(স্লাইডটাইপ পড়ুন: 07:33)

এখন যে চাইল্ড প্রসেসের proc গঠনটি সম্পূর্ণরূপে পূরণ করা হয়েছে, স্টেটটি EMBRYO থেকে RUNNABLE এ সুইচ করা হয়েছে। সুতরাং, runnable স্টেটে সেট করতে সিডিউলার আসলে এই চাইল্ড প্রসেসটিকে নির্বাচন করতে পারে এবং CPU টে বরাদ্দ করতে পারে। এইভাবে চাইল্ড প্রসেস CPU তে চালাতে এবং কোড এক্সিকিউট করতে সক্ষম হবে।

(স্লাইডটাইপ পড়ুন: 07:58)

ফর্ক বাস্তবায়ন থেকে pid এর মান ফেরত আসে, pid এর মান এখানে সেট করা আছে। সুতরাং, pid হল np pid এটি চাইল্ড প্রসেস এর pid মান। এবং এই pid র মান প্যারেন্ট প্রসেসের ফর্কের মধ্যে ফেরত আসে। এইভাবে, ইউজার স্পেসের প্যারেন্ট প্রসেসে, ফর্কটি চাইল্ড প্রসেসের pid ভ্যালু কে ফিরিয়ে দেয়, যখন আমরা এই পদ্ধতিতে চাইল্ড প্রসেসে এই লাইনটি দেখছি, ফর্কটি 0 এর মান নিয়ে ফিরে আসবে।

(স্লাইডপড়ুন সময়: 08:35)

যখন এটি CPU র রেজিস্টারে আসে, তখন চাইল্ড প্রসেসের সমস্ত রিকভারি মানগুলি দুটি রেজিস্টার ব্যতীত প্যারেন্ট প্রসেসের মতই হয়। আমরা আগে দেখেছি, একটি হল eax রেজিস্টার। সুতরাং, চাইল্ড প্রসেসে eax রেজিস্টারটি 0 তে সেট করা আছে যাতে ফর্ক যখন ফেরত আসে, এটি চাইল্ড প্রসেসে 0 এর মান নিয়ে আসে। এবং অন্য যে জিনিসটি চাইল্ড প্রসেসে পরিবর্তিত হয় তা হল eip বা নির্দেশ পয়েন্টার। সুতরাং নির্দেশ পয়েন্টার forkret এ সেট করা হয় যা একটি ফাংশন যা শুধুমাত্র চাইল্ড প্রসেস দ্বারা এক্সিকিউট হয় প্যারেন্ট দ্বারা নয়। সুতরাং, forkret কে xv6 কোডের একটি ফাংশন হিসাবে আপনি দেখতে পারেন।

(স্লাইডটাইপ দেখুন: 09:25)

এখন, আমরা আবার দেখবো exit সিস্টেম ইন্টারনালগুলিকে কল করে। সুতরাং, যখন একটি exit সিস্টেম কল এক্সিকিউট করা হয়, এই অপারেটিং সিস্টেমের ভিতরে ছয়টি জিনিস ঘটে। প্যারেন্ট, সব থোলা ফাইলের ব্যবহারের সংখ্যা হ্রাস হবে। উপরন্তু, যদি ব্যবহার সংখ্যা 0 তে যায় তবে ফাইলটি বন্ধ হয়ে যায়। দ্বিতীয়, সমস্ত ইন-মেমরি ইনডেক্সের জন্য রেফারেন্স একটি ড্রপ রয়েছে। তৃতীয়ত, প্যারেন্ট প্রসেসটিকে পাঠানো হয় একটি জাগরণ সংকেত বা wake up signal; প্যারেন্ট স্টেট ঘুমন্ত হয় তাহলে প্যারেন্টটি runnable হয়। কেন এই প্রয়োজন? এটা অবশ্যই প্রয়োজন কারণ একটি wait সিস্টেম কলের কারণে একে ঘুমাতে হতে পারে এবং সেইজন্য, এটি রানলেবল দ্বারা নিশ্চিত করা হবে যে wait সিস্টেম কল আনলকড রয়েছে। চতুর্থ টি হল প্রস্থান প্রসেস init তৈরি করবে, মনে রাখবেন যে init হচ্ছে প্রথম প্রসেসটি যেটি OS দ্বারা তৈরি করা হয়েছে। সুতরাং init প্রসেসকে প্রস্থান প্রসেসের সমস্ত চাইল্ডদের দত্তক গ্রহণ করা হয়, এবং প্রস্থান প্রসেস একটি ZOMBIE স্টেট এ সেট হয় বলা যেতে পারে। ZOMBIE স্টেটে এই সেটিংটি ব্যবহার করা হয়, যাতে প্যারেন্ট প্রসেসটি সেটিকে নির্ধারণ করবে যে

তার একটি চাইল্ড প্রসেস প্রস্থান করছে। সুতরাং, আমরা wait সিস্টেম কল সম্পর্কে আরো দেখব। এবং শেষ পর্যন্ত এটি সিডিউলারে একটি কন্টেক্সট সুইচ হবে।

(স্লাইডটাইম পড়ুন: 11:00)

আমরা পরবর্তী wait সিস্টেম কল দেখব। মনে রাখবেন যে যখন প্যারেন্টে wait সিস্টেম কল চালু করা হয়, এটি তার চাইল্ড প্রসেসের exit হওয়ার সময় পর্যন্ত এটি ব্লক করা হবে। সুতরাং, যদি কোনো চাইল্ড exit না করে তাহলে এটি অবরুদ্ধ থাকবে। অন্য দিকে, যদি প্যারেন্ট প্রসেসে কোনও চাইল্ড না থাকে, তাহলে এটি -1 সহ ফেরত আসবে। এখন আমরা wait সিস্টেম কল এর অভ্যন্তরীণ রূপটি দেখব।

(স্লাইডটাইপ পড়ুন: 11:30)

সুতরাং এটি xv6 অপারেটিং সিস্টেমে wait সিস্টেম কল এর বাস্তবায়ন। সুতরাং, এই তালিকাটি xv6 উৎস কোড এর proc.c ফাইল থেকে প্রাপ্ত করা হয়। আপনি দেখতে পাবেন যে এটির একটি অসীম লুপ আছে এই সেমিকোলনের জন্য, সেমিকোলন যা এখানে শুরু করা হয় এবং এই পয়েন্টে শেষ হয়। সুতরাং, এই বিশেষ অসীম লুপের ভিতরে, এখানে ভিতরে একটি for লুপ আছে যা এখান থেকে এখানে এসেছে। সুতরাং, এই ইনার ফর লুপ parses ptable এর মাধ্যমে। সুতরাং, মনে রাখবেন যে ptable প্রসেসের একটি অ্যারে হয়; এবং xv6 অপারেটিং সিস্টেমে প্রতিটি প্রজেক্ট ptable এর একটি এন্ট্রি আছে। সুতরাং, ptable এর সমস্ত উপাদান মাধ্যমে পার্সিং দ্বারা, এই বিশেষ লুপ xv6 OS এর মধ্যে উপস্থিত যে প্রত্যেক প্রসেস চেক করতে সক্ষম হবে। তাই প্রথম চেক যেটি করা হয় তা খুঁজে বের করতে হবে যে বর্তমান প্রসেসটি ptable এর মধ্যে এই বিশেষ এন্ট্রির প্যারেন্ট কী না, যা p হয়। সুতরাং, এটি অনুসন্ধান করা হচ্ছে যদি বর্তমান প্রসেসটি যা wait কে ইনভোক করেছে তা p এর প্যারেন্ট কী না। যদি এটি প্যারেন্ট না হয় তবে এটি চলতে থাকে; অন্যথায় এটি এখানে আসে। সুতরাং, এই বিশেষ সময়ে, বাস্তবায়নে, আমরা নিশ্চিত যে p একটি প্রসেস যার একটি ফর্ক চালু করে। পরবর্তী চেক p এর অবস্থা নির্ধারণ করে। সুতরাং, স্টেটটি যদি ZOMBIE হয়, এটি নির্দেশ করে যে চাইল্ড প্রসেসটি প্রস্থান করেছে, এবং সেইজন্য, এটি এই if কন্ডিশনে প্রবেশ করবে এবং এটি বিভিন্ন মুক্ত করার কাজগুলি করে যেমন এটি কার্নেল স্ট্যাকটি মুক্ত করে, স্টেটটি সেট করবে unused বলে, pid 0 তে সেট করবে, ইত্যাদি। এই বিন্দুতে প্রত্যাবর্তনটি, এই অসীম লুপ break এর ফলে এটিকে exit করার জন্য অপেক্ষা করতে হবে এবং pid রিটার্ন করবে; এখানে pid হল চাইল্ড প্রসেস এর pid মান।

(স্লাইডসময় পড়ুন: 14:00)

তাই, আপনি দুটি জিনিস লক্ষ্য করেছেন; প্রথমে exit করা চাইন্ডের কার্নেল স্ট্যাকটি সাফ করা হয় বা এই বিশেষ বিন্দুতে মুক্ত করা হয়, দ্বিতীয়টি অব্যাহতিপ্রাপ্ত চাইন্ডের সাথে সংশ্লিষ্ট পেজ নির্দেশিকাটি এই বিশেষ স্টেটমেন্টে মুক্ত। সুতরাং, এই চাইন্ড প্রসেস স্ট্যাকের পাশাপাশি পেজ ডিরেক্টরিটি মুক্ত করলে প্যারেন্ট প্রসেসকে ছেড়ে যাওয়া চাইন্ড প্রসেসের মধ্যে নিমজ্জিত করবে। সুতরাং, এটি চাইন্ড প্রসেসের ভাল ডিবাগিং সুবিধাগুলি সক্ষম করে। উদাহরণস্বরূপ, যদি চাইন্ডটি ক্র্যাশ হয়ে যায় তবে প্যারেন্ট প্রসেস স্ট্যাকের পাশাপাশি পেজ ডিরেক্টরিটি সন্ধান করতে পারে, এবং সেইজন্য চাইন্ড প্রসেসের ফিজিক্যাল পেজগুলির মধ্যে এবং আমরা চাইন্ড প্রসেসটি সম্পর্কে তথ্য পেতে সক্ষম হব যে কেন এটা ক্র্যাশ করেছে।

(স্লাইডটাইপ পড়ুন: 14:54)

পরবর্তী পুরো for লুপের জন্য, যদি আমরা কোনও বিশেষ প্রসেসের জন্য কোন চাইন্ড খুঁজে না পাই তাহলে আমরা কেবলমাত্র 1 বিয়োগ ফিরণ পাঠাবো। সুতরাং, wait আবার ব্যবহারকারীর প্রসেসে ফিরে আসবে, তবে তার মান হবে -1.

(স্লাইডটাইপ পড়ুন: 15:10)

সুতরাং, এই বিশেষ লাইনটিতে এক্সিকিউশন হবে অর্থাৎ sleep হবে, যখন আমাদের একটি চাইন্ড প্রসেস আছে যা জুমবিই অবস্থায় নেই। সুতরাং, এই ক্ষেত্রে এই wait বিবৃতি sleep এ চলে যাবে যতক্ষণ না পর্যন্ত এটি একটি exit চাইন্ড দ্বারা জাগ্রত হয়।

(স্লাইডটাইপ পড়ুন: 15:28)

তাই আমরা এখন exec সিস্টেম কল এর ইন্টার্নাল (internal) এর দিকে তাকাব। মনে রাখবেন যে exec সিস্টেম কল মেমরিতে একটি প্রোগ্রাম লোড করে এবং তারপর এটি এক্সিকিউট হয়। এই বিশেষ স্বতন্ত্র কোডের মধ্যে, প্যারেন্ট প্রসেস ফর্ক সিস্টেম কল আহ্বান করবে যার ফলে একটি চাইন্ড প্রসেস তৈরি হবে। এই বিশেষ উদাহরণে চাইন্ড প্রসেস exec সিস্টেম কল এক্সিকিউট করবে; এবং এর ফলে, একটি নতুন প্রোগ্রাম সিস্টেমের মধ্যে এক্সিকিউট হবে। এই বিশেষ উদাহরণে, নতুন প্রোগ্রাম 'ls' প্রোগ্রাম, যা বর্তমান ডিরেক্টরীতে সমস্ত ফাইলের তালিকা করে। এখন, 'ls' একটি এক্সিকিউটেবল। এটির বর্তমান একটি নির্দিষ্ট বিন্যাস থাকে যা ELF বিন্যাস বা এক্সিকিউটেবল লিঙ্কার ফরম্যাট হিসাবে পরিচিত। সুতরাং, এই নির্দিষ্ট ফরম্যাটটি অপারেটিং সিস্টেমের মধ্যে exec সিস্টেম কল দ্বারা অভ্যন্তরীণভাবে ব্যাখ্যা করা হয় এবং এই বিন্যাস হার্ড ডিস্ক থেকে RAM র মধ্যে 'ls' লোড করতে ব্যবহৃত হয়।

(স্লাইডটাইপ পড়ুন: 16:41)

তাই আসুন দেখি ELF ফরম্যাটে আসলে কি উপস্থিত আছে। সুতরাং, প্রত্যেক সময় আমরা আসলে একটি প্রোগ্রাম কম্পাইল বা লিঙ্ক করি, এটি ELF এক্সিকিউটেবল বা একটি ELF অবজেক্ট বা বস্তু তৈরি করে। উদাহরণস্বরূপ, আমরা সেটি গ্রহণ করেছি - স্ল্যাশ বিন স্ল্যাশ ls, এটি একটি ELF এক্সিকিউটেবল। সুতরাং, এটি একটি বিন্যাস যা এখানে দেখানো হয়েছে, তাই অন্তত এই বিন্যাসের অংশ, এবং বিন্যাস সম্পর্কে আরও বিস্তারিত এই বিশেষ রেফারেন্স থেকে পাওয়া যাবে। আমরা এখন কি করব তা হল আমরা এই ELF এক্সিকিউটেবলের কিছু গুরুত্বপূর্ণ উপাদানগুলির মধ্য দিয়ে যাব এবং দেখব যে এটি আমাদের জন্য যে কোনও অপারেটিং সিস্টেমে ব্যবহার করা যায়।

(স্লাইডটাইপ পড়ুন: 17:19)

সুতরাং, আমরা ELF হেডার দিয়ে শুরু করব। ELF হেডারটির বিভিন্ন প্যারামিটার রয়েছে এবং এইগুলির মধ্যে এখানে কয়েকটি প্যারামিটার উপস্থিত রয়েছে। সুতরাং ELF হেডার একটি আইডেন্টিফাইয়ারের সাথে শুরু হয়। সুতরাং, এই সনাক্তকারীটি একটি জাদুর সংখ্যা যা এই ফাইল প্রকৃতপক্ষে একটি ELF ফাইল কিনা তা সনাক্ত করতে ব্যবহৃত হয়। দ্বিতীয়, টাইপ আছে তাই টাইপ এই ELF ফাইল টাইপ বলতে পারে, তাই টাইপের এক্সিকিউটেবল বা relocatable বস্তুর বা শেয়ার্ড বস্তু বা কোর ফাইল ইত্যাদির মান থাকে। আরেকটি গুরুত্বপূর্ণ এন্ট্রি হল মেশিন বিবরণ। তাই এটি এই ELF এক্সিকিউটেবল বা ELF বস্তুটি একটি নির্দিষ্ট মেশিনে চালাতে পারবে কি না তা সম্পর্কে তথ্য জানাবে। উদাহরণস্বরূপ, ELF এই মেশিনের বিশদ যেমন i386, x86_64, এআরএম (ARM), এমআইপিএস(MIPS) এবং এর মতো মান থাকতে পারে। তারপর আমাদের ELF হেডারের একটি এন্ট্রি মান আছে, এই এন্ট্রি মান প্রোগ্রামটি এক্সিকিউট কোথায় শুরু করা উচিত তার ভার্চুয়াল ঠিকানা বলবে। সুতরাং, পাশে আমরা প্রোগ্রাম হেডারের একটি পয়েন্টার আছে, উপস্থিত প্রোগ্রাম হেডারের সংখ্যা, সেকশন হেডার পয়েন্টার এবং বর্তমান সেকশন হেডারের সংখ্যা। সুতরাং, আমরা পরবর্তী স্লাইডে এই সম্পর্কে আরো দেখতে পাব।

(স্লাইডটাইপ পড়ুন: 18:44)

সুতরাং আসুন প্রথমে একটি উদাহরণ দিয়ে শুরু করা যাক। তাই আসুন আমরা এই C তে লেখা হ্যাঁলো ওয়ার্ল্ড প্রোগ্রামের এই উদাহরণটি বিবেচনা করি, এবং আমরা এটি - c দিয়ে কম্পাইল করি। সুতরাং, আপনি যখন gcc হ্যাঁলো ডট C -c বলছেন, এটি বস্তু ফাইল হ্যাঁলো ডট ওপেন তৈরি করে। সুতরাং, এটি একটি ELF অবজেক্ট, তারপর আমরা একটি বিকল্প ব্যবহার করে হ্যাঁলো ডট ও অবজেক্ট ফাইলের সঙ্গে বিকল্প -H ব্যবহার করি। তাই -H বিকল্প ELF হেডারটি মুদ্রণ করবে। সুতরাং, এটি ELF হেডার, তাই এটির একটি জাদু সংখ্যা যা আইডেন্টিফায়ার এবং এটি অন্য কোন ফাইল থেকে এই নির্দিষ্ট বস্তুর ফাইলটি আলাদা করার জন্য ব্যবহৃত হয়। সুতরাং, এটি বলা হয় যে এই নির্দিষ্ট অবজেক্ট ফাইল প্রকৃতপক্ষে একটি ELF ফাইল। তারপর আরেকটি জিনিস যা

আমরা দেখেছি তা হল ELF হেডারের প্রকার, এটি উদাহরণস্বরূপ একটি স্থায়ী বস্তু এবং সেইজন্যে। সুতরাং, এই বিশেষ উদাহরণে, যেহেতু আমরা একটি হ্যালো ডট ও অবজেক্ট ফাইলে একটি ডট ব্যবহার করি, এটি একটি রিলকেটেবল অবজেক্ট ফাইল এবং আপনি এটি এখানে REL কে দেখুন এটি একটি relocatable অবজেক্ট ফাইল। এবং আরেকটি বিষয় হলো মেশিনের বিবরণ, যা এখানে মেশিন হিসাবে নির্দিষ্ট করে এই ক্ষেত্রে এটি AMD x86-64। সুতরাং, যা নির্দেশ করে যে এই অবজেক্টের ফাইলটি x86-64 বিট মেশিনের জন্য, তারপর আমরা এন্ট্রি পয়েন্টটি দেখেছি তা হল 0। তাই, আমরা যা দেখেছি তা অন্য কিছু যা প্রোগ্রাম হেডারগুলির শুরু হয় 0 থেকে (বাইটের মধ্যে ফাইল), এই একটি পয়েন্টার থেকে প্রোগ্রাম হেডার। এবং আমরা এই নির্দিষ্ট অবজেক্ট ফাইলে প্রোগ্রাম হেডার সংখ্যা দেখেছি 0। অন্য দুটি দিকের মধ্যে সেকশন হেডারের পয়েন্টার এবং সেকশন হেডারের সংখ্যা। সুতরাং সেকশন হেডারের পয়েন্টার হল সেকশন হেডারের শুরু যেটা 368, এবং সেকশন হেডারের সংখ্যা 13। সুতরাং, এইভাবে, আমরা আসলে ELF হেডারের বিভিন্ন সামগ্রী দেখতে পারি।

(স্লাইডটাইপ পড়ুন: 20:49)

এখন, আমরা সেকশন হেডার সম্পর্কে আরো বিস্তারিতভাবে দেখব, সেকশন হেডারের তালিকা পেতে হলে আপনি readelf থেকে এস হ্যালো ডট ও বিয়োগ করতে পারেন, যা রিলকেটেবল অবজেক্ট হ্যালো ডট ও তে উপস্থিত সমস্ত সেকশন হেডারকে মুদ্রণ করবে। সুতরাং, এটাই আসলে এখানে দেখানো হয়। তাই, আমরা এই সম্পর্কে আরও বিস্তারিত জানাই না কারণ আমাদের জন্য অনেক বেশি প্রযোজ্য নয়, তবে বিভিন্ন কলাম সম্পর্কে একটি ব্যাখ্যা দেওয়া হল। সুতরাং, এখানে এই দ্বিতীয় কলামটি হল এই বিশেষ কলামটি সেকশনগুলির নাম, তৃতীয় কলামটি আপনাকে সেকশনগুলির ধরণ প্রদান করে। সুতরাং এগুলো যে কোনও প্রকারের হতে পারে, এটি হল PROGBITS যা প্রোগ্রাম দ্বারা সংজ্ঞায়িত তথ্য, SYMTAB যা একটি সিম্বল টেবিল, NULL বা নোবিট একটি অংশ যা কোন বিট ধরে না এবং RELA যা একটি রিলোকেটেবল টেবিল। তারপর আমরা এখানে ঠিকানা যা একটি ভার্সিয়াল ঠিকানা যেখানে সেকশন লোড করা উচিত। এই বিশেষ ক্ষেত্রে, এটি সব 0 এর কারণ এটি একটি ডট ও ফাইল, এটি একটি অবজেক্ট ফাইল এবং এটি স্থানান্তরিত করা যেতে পারে। তারপর এখানে আপনার অফসেট এবং সেকশনগুলির আকার আছে, এখানে একটি টেবিলের আকার আছে যদি আপনার একটি টেবিলের থাকে তাহলে আপনি একটি শূন্য নয় এমন মান আছে; যাইহোক, যদি কোন টেবিল উপস্থিত না থাকে তবে আপনার একটি 0 মান আছে।

(স্লাইডটাইপ পড়ুন: 22:11)

এর পরে, আমরা প্রোগ্রাম হেডার কনটেন্টস দেখব, প্রোগ্রামের হেডারের বিভিন্ন প্যারামিটার রয়েছে যেমন প্রোগ্রাম হেডারের ধরন, অফসেট, ভার্চুয়াল ঠিকানা যেখানে সেগমেন্ট লোড করা প্রয়োজন, এবং আপনার একটা p অ্যাড্রেস অফসেট থাকে যা উপেক্ষা করা হয়।

(স্লাইডটাইপ পড়ুন: 22:34)

সুতরাং, আমাদের হ্যালো ওয়ার্ল্ড প্রোগ্রামের জন্য প্রোগ্রাম হেডার পেতে, তাই আমরা readelf বিয়োগ l ব্যবহার করি। তাই লক্ষ্য করুন, আমরা এখানে এক্সিকিউটেবল প্রদান করছি এবং অবজেক্ট নয়। সুতরাং এই প্যারামিটারগুলি যা প্রোগ্রাম হেডার সম্পর্কিত মুদ্রিত হয়। সুতরাং, আমাদের আছে বিভিন্ন ধরনের হেডার, অফসেট, ভার্চুয়াল ঠিকানা যেখানে এটি লোড করা প্রয়োজন, ফিজিক্যাল ঠিকানা, মেমোরি আকার এবং ক্লাসস।

(স্লাইডসময় পড়ুন: 23:05)

এখন আমাদের ELF এক্সিকিউটেবল ফাইল এবং exec elf বস্তু সম্পর্কে কিছু ধারণা আছে। তাদের কীভাবে সংরক্ষণ করা হয় এবং ELF ইমেজের বিভিন্ন উপাদানগুলি, আমরা দেখতে পাই যে xv6 OS এ কীভাবে exec সিস্টেম কল প্রয়োগ করা হয়। সুতরাং, এই আসলে এখানে তালিকাভুক্ত করা হয়। এবং আপনি এটি xv6 সোর্স কোডের এই বিশেষ ফাইলের মধ্যেও দেখতে পারেন, যেটি exec dot c। সুতরাং, স্লাইডে দেখানো হয়েছে ভার্চুয়াল মেমোরি ম্যাপ। সুতরাং, চাইল্ড প্রসেস দ্বারা exec সিস্টেম কল ইনভোক করা হয় তা স্মরণ করুন। তাই প্রথমত, একটি প্যারেন্ট প্রসেস আছে যা ফর্ক করে এবং চাইল্ড প্রসেসে, exec সিস্টেম কল যা চালু হয়। সুতরাং, যখন চাইল্ড প্রসেসকে ফর্কিং দ্বারা তৈরি করা হয়, তখন একটি ভার্চুয়াল মেমরি ম্যাপ তৈরি হয় যা চাইল্ড প্রসেসের জন্য তৈরি করা হয়। এবং আমরা উপরের অর্ধেক বা ভার্চুয়াল মেমরির ম্যাপের শীর্ষস্থানীয় অঞ্চলগুলি দেখেছি যেমনটি কার্নেল কোডের অন্তর্গত, আবার সর্বনিম্ন অর্ধেক ব্যবহারকারীর কোড এবং ডেটা নিয়ে গঠিত। এছাড়াও আমরা দেখেছি যে, ফর্কের প্রসেস চলাকালীন কার্নেল স্ট্যাক তৈরি হয়। সুতরাং, এই কার্নেল স্ট্যাকটি এই চাইল্ড প্রসেসের জন্য নির্দিষ্ট। এখন, আমরা কীভাবে exec ফাংশন এই ভার্চুয়াল মেমরি ম্যাপ পরিবর্তনকে এক্সিকিউট করে তা দেখব। সুতরাং, শুরু করা exec প্যারামিটারগুলি বিবেচনা করা যাক। সুতরাং, এই ক্ষেত্রে দুটি প্যারামিটার নেওয়া হয় যা পাথ এবং argv হয়, তাই পাথ এক্সিকিউটেবলের পাথ নির্দিষ্ট করে। সুতরাং, আমাদের উদাহরণে আমরা স্ল্যাশ বিন স্ল্যাশ ls ব্যবহার করেছি। সুতরাং, এই পথের মধ্যে নির্দিষ্ট করা হবে যখন argv হল প্যারামিটার যা নির্দিষ্ট প্রোগ্রামে পাশ করা হয়। উদাহরণস্বরূপ, যদি আপনি গ্রহণ করেন তবে ls বিয়োগ l অথবা ls বিয়োগ t এর মতো আর্গুমেন্ট থাকতে পারে। সুতরাং মাইনাস এল এবং মাইনাসটি হল আর্গুমেন্ট যা এখানে পাস করা হয়।

(স্লাইডটাইপ পড়ুন: 25:16)

তাই প্রথম কাজটি করা হয় যে আমরা এক্সিকিউটেবলের inode একটি পয়েন্টার পাই। এখন inodeটি একটি মেটাডেটা যেটি সম্পর্কে তথ্য রয়েছে যে সেকেন্ডারি স্টোরে যেমন হার্ড ডিস্কের কোন অংশে ঐ বিশেষ এক্সিকিউটেবলটি সঞ্চিত রয়েছে। উদাহরণস্বরূপ, আমাদের বিশেষ ক্ষেত্রে, যেখানে আমরা গ্ল্যাশ বিন গ্ল্যাশ ls ব্যবহার করছি, তাই এই বিশেষ ফাংশন নামটি ls এক্সিকিউটেবলের জন্য inode এ ফিরে আসবে। পরবর্তী, আমরা যা করছি তা হচ্ছে আমরা এই ফাংশনটি ব্যবহার করছি যাটি readi নামে পরিচিত, যা এই inode থেকে পড়েছে যা সেকেন্ডারি স্টোরেজ ডিভাইস থেকে এসেছে, ELF হেডার। সুতরাং, আমাদের এই বিশেষ inode থেকে ELF হেডার পড়া হয়। সুতরাং, ELF হেডার বা শুধু এখানে elf উল্লেখ হিসাবে elf হেডার হিসাবে এখানে সংজ্ঞায়িত করা হয়। সুতরাং, আমরা এই ELF হেডারের দিকে নজর রাখি এবং শনাক্তকারীকে পরীক্ষা করে দেখি, আমরা ম্যাজিক নম্বর পরীক্ষা করছি এবং আমরা যাচাই করছি যে এই জাদু সংখ্যা সঠিক কিনা।

(স্লাইডসময় পড়ুন: 26:23)

সুতরাং, এটি একটি স্যানিটি চেক এবং ELF এর জন্য এই জাদুকর সংখ্যাটির এই বিশেষ মান থাকা উচিত, এটি হল 7fELF থাকা উচিত। সুতরাং, এই 7f হেক্সাডেসিম্যাল মান, যখন ELF হল অক্ষর।

(স্লাইডসময় পড়ুন: 26:46)

পরবর্তী ধাপ setupkvm একটি কল, যেখানে আমরা পুনরায় কার্নেল পার্মপেজ টেবিলের সেটআপ। সুতরাং এটা প্রয়োজন নাও হতে পারে, কিন্তু এটি xv6 এর মধ্যে সম্পন্ন করা হয়, যদিও এটি একটি প্রয়োজনীয়তা নয়, আমরা ইতিমধ্যে ফর্ক প্রসেস সময় এটি করেছি।

(স্লাইডটাইপ পড়ুন: 27:07)

সুতরাং এই বিশেষ স্লাইড exec সম্পাদনের ধারাবাহিকতা প্রদর্শন করে। তাই আমরা inode থেকে পড়া চালিয়ে যাচ্ছি। আমরা প্রোগ্রাম হেডারগুলির মত বিভিন্ন জিনিসগুলি পড়ি এবং আমরা ELF ইমেজ থেকে কোড এবং ডেটা লোড করা শুরু করি যা হার্ড ডিস্কে RAM এ উপস্থিত থাকে, এবং ফলস্বরূপ আমরা আসলে কোড এবং ডেটা অনুসারে ভার্চুয়াল অ্যাড্রেস স্পেস পূরণ করছি। সুতরাং, এটাই এখানে সম্পন্ন করা হয়। তাই আমি এই সম্পর্কে আরও বিস্তারিত বলব না যে কিভাবে বিভিন্ন ফাংশন ব্যবহার করা হয়। কিন্তু মৌলিক ধারণা হল যে, আমরা হার্ড ডিস্কের মধ্যে inode কে দেখবো সংশ্লিষ্ট নির্দিষ্ট এক্সিকিউটেবল এবং কোডটি লোড করে এবং ফিজিক্যাল মেমোরি ম্যাপে শুধুমাত্র পাঠ্য ডাটা এবং আরও আমরা তৈরি করেছি পেজ টেবিল এবং পেজ ডিরেক্টরী এন্ট্রিগুলিও সংশ্লিষ্ট কোড এবং তথ্য। সুতরাং, এটি আসলে উপস্থিত, তাই, আপনি এখানে এই ম্যাপিং পাবেন।

(স্লাইডসময় পড়ুন: 28:12)

exec বাস্তবায়নের পরবর্তী ধাপ ব্যবহারকারী প্রসেসের জন্য স্ট্যাক তৈরি করা। সুতরাং, এই স্ট্যাকের কার্নেল স্ট্যাকের বিপরীতে স্থানীয় ভেরিয়েবলগুলি সংরক্ষণ করার জন্য এবং ফাংশন কলগুলির জন্য কোড দ্বারা ব্যবহৃত হয়। সুতরাং, এই স্ট্যাক তৈরি করার জন্য, আমরা বরং exec বাস্তবায়ন দুটি সন্নিহিত পেজ বরাদ্দ করে। সুতরাং, একটি স্ট্যাকের জন্য ব্যবহার করা হয় আর অন্যটি রক্ষক বা গার্ড পেজ হিসাবে ব্যবহার করা হয়। তাই গার্ড পেজ অপ্ৰবেশ্য হয়; এই স্ট্যাক ওভারলোপগুলি রক্ষা করার জন্য এটি ব্যবহৃত হয়। সুতরাং, এর অর্থ কি যে আমরা স্ট্যাকটি ব্যবহার করে থাকি, স্ট্যাকের আকার বাড়ছে এবং অবশেষে গার্ড পেজকে আঘাত করবে; এবং ফলস্বরূপ, আমরা জানি যে স্ট্যাকের ওভারলোপ ঘটেছে।

(স্লাইডটাইপ দেখুন: ২৭:০৭)

exec বাস্তবায়নের পরবর্তী ধাপ ব্যবহারকারী স্ট্যাক পূরণ করা। সুতরাং, আমরা এখানে স্ট্যাক তৈরি করেছি এবং এখন আমরা আসলে স্ট্যাক ভরাট করছি। সুতরাং আমরা কমান্ড লাইন আর্গুমেন্টের সঙ্গে স্ট্যাক পূরণ করব। সুতরাং, আমরা জানি যে আমাদের লেখা কোন প্রোগ্রাম কমান্ড লাইন আর্গুমেন্ট নিতে পারে এবং এই আর্গুমেন্টগুলি আসলে স্ট্যাকের মধ্যেটা ভরতি করে। সুতরাং, আমাদের একটি কমান্ড লাইন আর্গুমেন্ট 0 থেকে N আছে, একটি নাল পরিসমাপ্তি স্ট্রিং দ্বারা অনুসরণ করা আছে এবং তারপর আমাদের আছে আর্গুমেন্টগুলোর পয়েন্টার অর্থাৎ আর্গুমেন্ট 0 এর পয়েন্টার আছে, আর্গুমেন্ট 1 এর পয়েন্টার, ইত্যাদি। সুতরাং এই আর্গুমেন্ট এর পয়েন্টার, আপনার প্রোগ্রামের argv ফর্ম। তাই, আপনি জানেন যে একটি মেইন ফাংশন argc এবং argv ব্যবহার করে, তাই এই পয়েন্টারগুলি প্রোগ্রাম ইনপুট এর argv আর তারপর এই argv পরে argc হয়, যা এই ধরনের প্যারামিটারের সংখ্যা এবং এর দ্বারা অনুসরণ করা হয়, এর জন্য একটি Dummy রিটার্ন অবস্থান আছে মেইনের। এখন, আমরা দেখেছি যে আমরা আসলে ব্যবহারকারীর প্রসেসের জন্য কোড তৈরি করেছি, আমাদের কাছে ব্যবহারকারীর প্রসেসের তথ্য আছে এবং এই দুটি আসলেই সেকেন্ডারি স্টোরেজ ডিভাইস থেকে নেওয়া হয়েছে এবং ফিজিক্যাল RAM এ লোড করা হয়েছে। এবং একটি পেজ ডিরেক্টরি ও এবং পেজ টেবিল এন্ট্রি তৈরি করা হয়েছে তাই আমরা ভার্চুয়াল প্রধান মেমরি ম্যাপে এটি দেখতে সক্ষম। এবং আমরা এই ব্যবহারকারীর প্রসেসের জন্য একটি স্ট্যাক তৈরি করেছি এবং আমরা স্ট্যাককে কমান্ড লাইন আর্গুমেন্ট দিয়ে ভরাট করেছি, সঠিকভাবে argc এবং argv তৈরি করেছি। পরবর্তী কাজটি শুধুমাত্র প্রসেসটি চালানো শুরু করা।

(স্লাইডটাইপ পড়ুন: 30:42)

তাই এই elf এন্ট্রি দিয়ে এই বর্তমান প্রসেস জন্য trapframe ভর্তি দ্বারা সম্পন্ন হয়। সুতরাং, tf eip যে trapframe মধ্যে নির্দেশ পয়েন্টার হয় elf ডট এন্ট্রি, যা ব্যবহারকারী প্রোগ্রামের প্রধান একটি পয়েন্টার বা ব্যবহারকারী

প্রোগ্রামের প্রধান ফাংশনের সঙ্গে সংরক্ষিত হয়। একইভাবে, স্ট্যাক পয়েন্টার sp তে সেট করা হয়, তাই আমরা trapframe esp তৈরি করছি যা SP তে সেট করা হয়। এখন যখন এই বিশেষ exec সিস্টেম কল ব্যবহারকারীর প্রসেসকে রিটার্ন করে, trapframe রেজিস্টারে মধ্যে পুনরুদ্ধার করা হয় ফলে eip মেইন ঠিকানা মান পায় যখন স্ট্যাক পয়েন্টার esp ব্যবহারকারী স্থান স্ট্যাক মান পায় এবং সেইজন্য, এক্সিকিউশন প্রধান প্রোগ্রাম থেকে শুরু হবে। স্ট্যাক পয়েন্টারের argc এবং argv এর জন্য বিভিন্ন আর্গুমেন্টের পয়েন্টার থাকবে, যা মেইন প্রোগ্রামে ব্যবহার করা হবে। তাই এই সাথে, আমরা এই ভিডিও বক্তৃতার শেষে আসি। আমরা ফর্ক, wait, exec এবং exit সিস্টেম কল যেমন প্রসেস পরিচালনার সাথে সম্পর্কিত বিভিন্ন সিস্টেম কল প্রয়োগ করে কিভাবে xv6 অপারেটিং সিস্টেম সম্পর্কে বেশ বিস্তারিতভাবে দেখেছি। সুতরাং, exec system call- এর সাথে, আমরা দেখেছি কিভাবে বিভিন্ন ব্যবহারকারীর স্থান সেশন যেমন কোড, ডেটা এবং স্ট্যাক তৈরি করে এবং কিভাবে স্ট্যাক কমান্ড লাইন আর্গুমেন্টগুলি দ্বারা পূরণ করে এবং কিভাবে ইউজার স্পেস প্রসেস শুরু হয়। ধন্যবাদ।